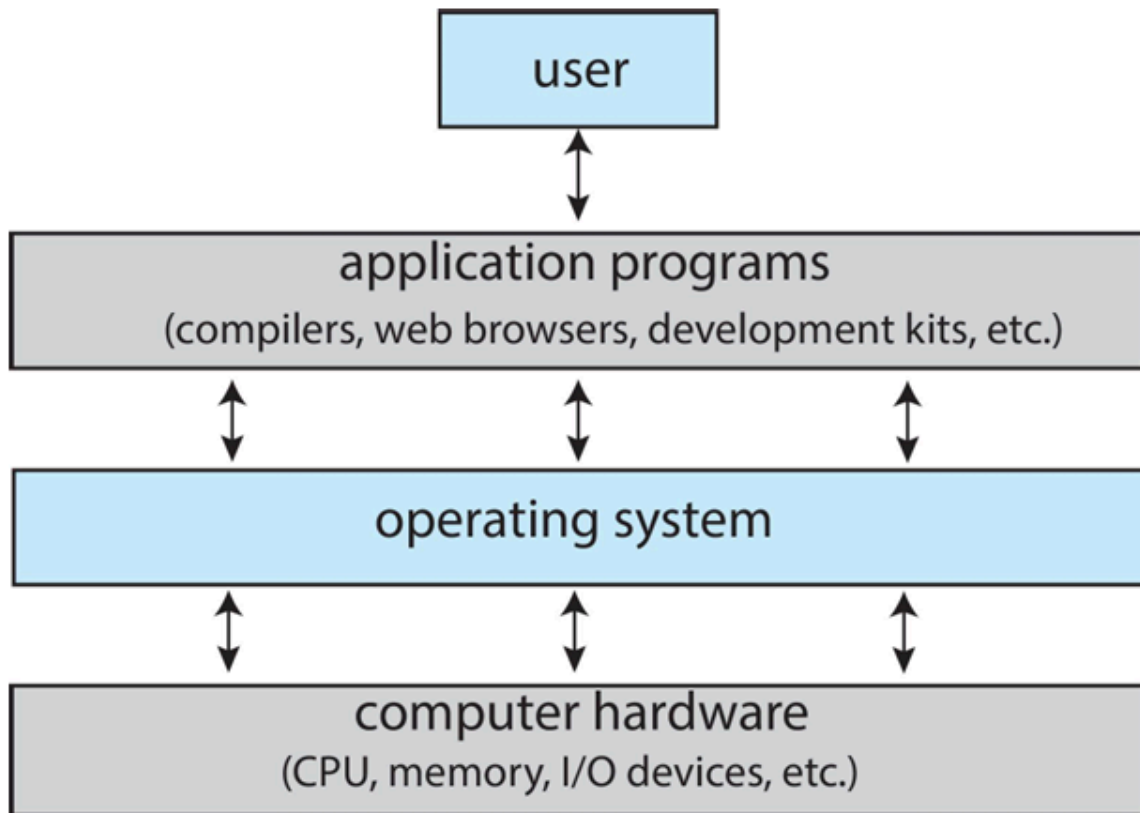




1intro

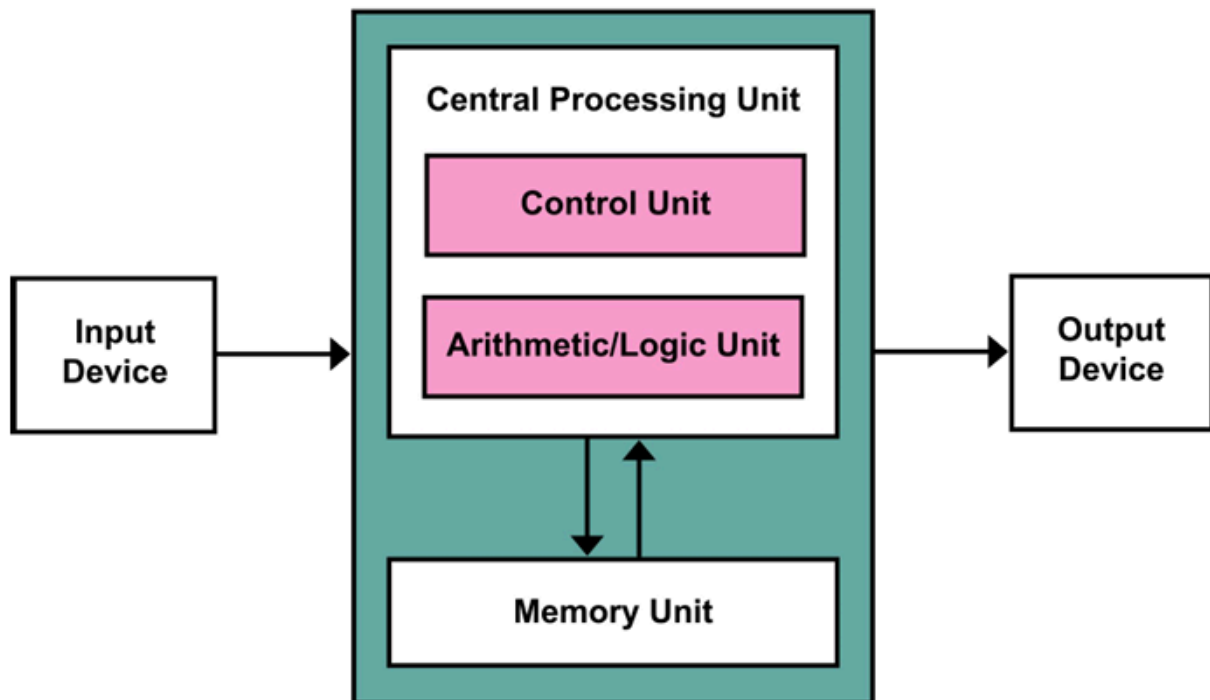
İşletim Sistemi Nedir?



Bir Program Nasıl Çalıştırılır?

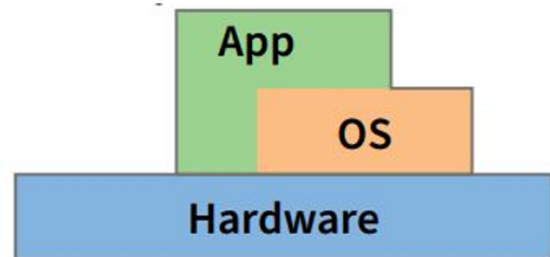
- Bir program çalıştırıldığında OS önce o programın kodlarını sabit diskten ram'e yükler.
- Daha sonra sırayla talimatları (komutlar) ram'den CPU'ya aktarılır.
- CPU bu komutları anlar ve işler.

Von Neumann Architecture:



Early-Primitive Operating Systems

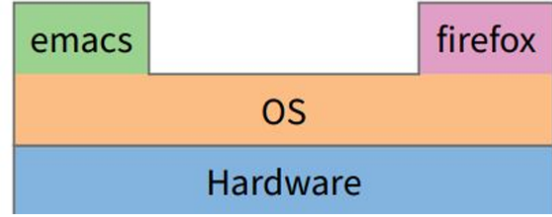
- Her seferinde tek bir program çalıştırılabilirdi. Eğer program I/O bekliyorsa CPU boş kalırdı ve diğer programlar mevcut programın işinin bitmesini beklemek zorunda kalırdı.
- Programları birbirinden ayıracak bir koruma mekanizması veya sanallaştırma yoktu.
- Kötü niyetli kullanıcılar ve hatalı programların (örn. sonsuz döngü) olmadığı varsayılırdı.



Multi-Tasking & Protection

- Kaynak yönetimi daha verimli hale getirildi.

- Bir program beklerken diğerleri CPU üzerinde kendi işlemlerini gerçekleştirebilmeye başladı.



- Hatalı programların (örn. sonsuz döngü) CPU'yu rehin alma olasılığına karşın OS'e programı durdurup CPU'yu elinden alma yetkisi verildi.
- Programların bellek alanları sanallaştırılarak bir programın başka bir programın bellek alanına müdahale etmesi engellendi.

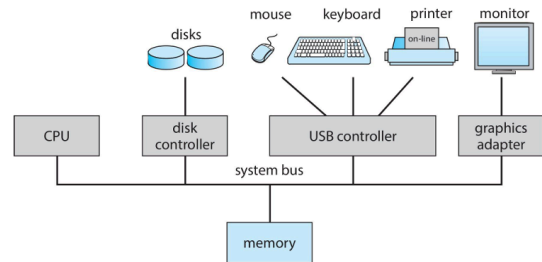
Multi-User OS

- Burada multi-tasking'te olan protection'ın user seviyesinde taşınması vardır.
- Sistem kaynakları kullanıcılar arasında verimli şekilde kullanılır.
 - Bellek talepleri sürekli %100 oranında değildir, bir kullanıcı CPU'yu kullanmadığında işlemcideki boşluk diğer kullanıcıya verilir.
- Bazı kullanıcılar veya uygulamalar kaynakları sınır tanımadan kullanmak isteyebilir. İşletim sisteminin bunu engellemek için adil paylaşım sağlayan katı politikalara ihtiyacı vardır.

Bilgisayar Sisteminin Temen Yapısı

1. Sistem Veriyolu

- a. CPU, disk (disk controller), mouse & keyboard & printer & etc. (USB controller), monitör (graphics adapter) gibi donanımların hepsi sistem veriyoluna (system bus) bağlıdır.



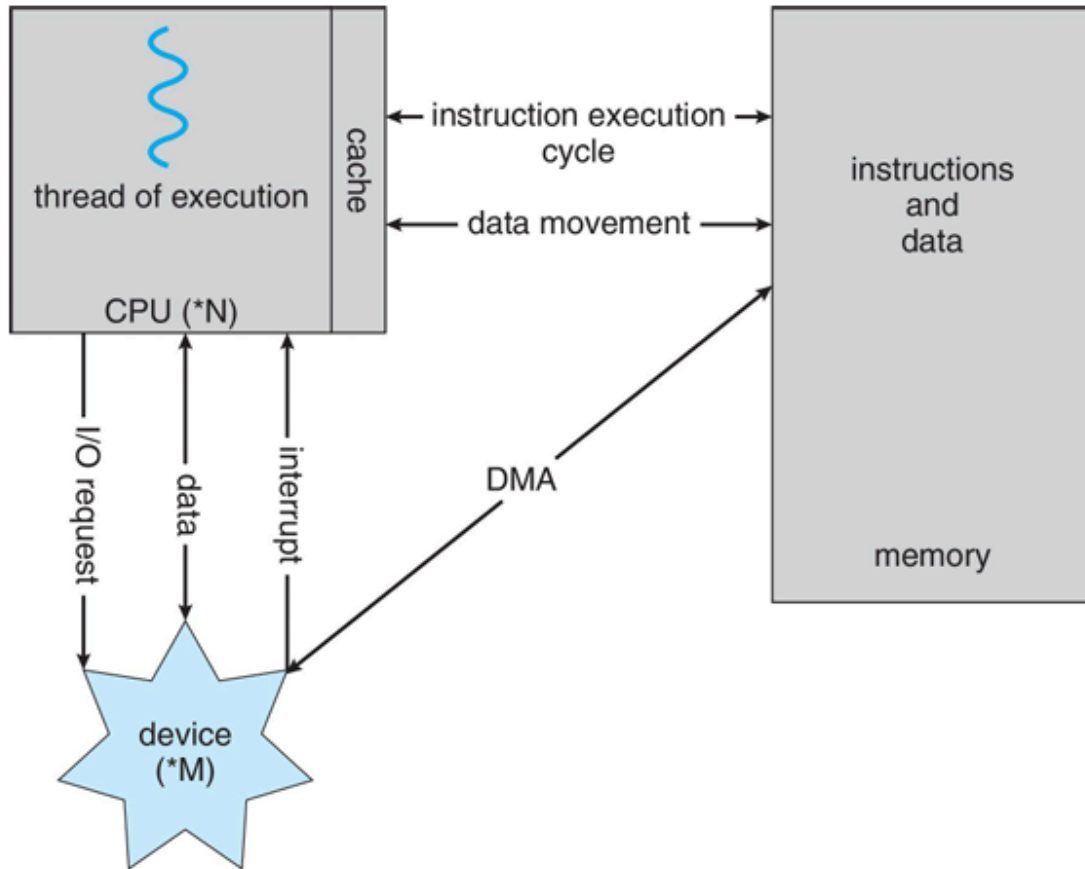
1. Paylaşımlı Bellek

- a. Veriyolunun en altında bellek bulunur ve bütün donanım cihazları verileri okumak ve yazmak için bu ortak belleğe erişim sağlar.

2. Eşzamanlılık ve Rekabet

- a. Bilgisayarda oyun oynarken (CPU kullanırken) aynı anda fareyi hareket ettiriyor (USB) ve dosya indiriyor (disk) olabilirsiniz.
- b. Bu donanımların hepsi eşzamanlı (concurrent) çalışır ve hepsi aynı anda belleğe ulaşmak için rekabet ederler.
- c. İşte işletim sisteminin (ve donanımdaki bellek denetleyicilerinin) en büyük görevi, bu rekabeti bir kaosa dönüşmeden sıraya koymak ve trafik kazası yaşanmasını engellemektir.

Modern Bir Bilgisayar Nasıl Çalışır?



CPU ve Bellek Arasındaki İletişim

- CPU'nun içindeki "iş parçacığı" (thread of execution) , bellekten (memory) sürekli komut ve veri çeker.

CPU ve Cihazlar Arasındaki İletişim

- CPU, diskten bir cihaz (device) üzerinden dosya okumak istiyor. Cihaza bir **I/O isteği (I/O request)** gönderir.

DMA (Doğrudan Bellek Erişimi)

- Cihaz çok büyük bir veri okuduğunda (örneğin 1 GB'lık bir video dosyası), bu veriyi (data) bayt bayt CPU üzerinden belleğe taşımak CPU'yu tamamen felç eder.
- DMA sayesinde cihaz, işlemciyi hiç meşgul etmeden veriyi **doğrudan belleğe (memory)** yazar. Veri taşıma işlemi bittiğinde CPU'ya sadece bir tane "İşim bitti" interrupt'ı yollar. Bu yöntem sistem performansını inanılmaz derecede artırır.

System Calls

Sıradan bir uygulamanın, işletim sisteminden (OS) kendi adına bir işlem yapmasını istemesine sistem çağrısı denir. İşletim sistemi adına çalışan bu kodlar son derece özeldir.

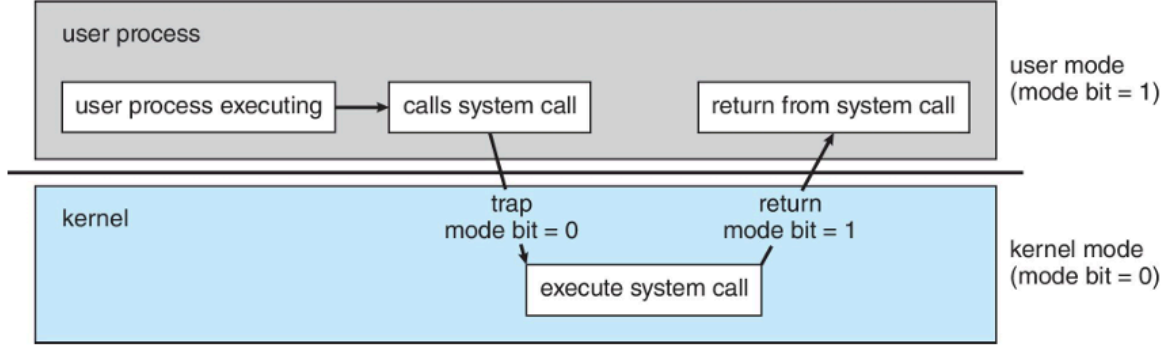
- İşletim sistemi rutinlerini sadece basit bir kütüphane fonksiyonu çağırır gibi çağıramazsın.
- İşletim sisteminin içine geçişi resmi ve son derece kontrollü bir sürece dönüştürmek için tasarlanmış özel donanım komutları eklenmiştir.

İkili Mod Operasyonları

İşletim Sisteminde kullanıcı ve çekirdek olmak üzere 2 mod bulunur.

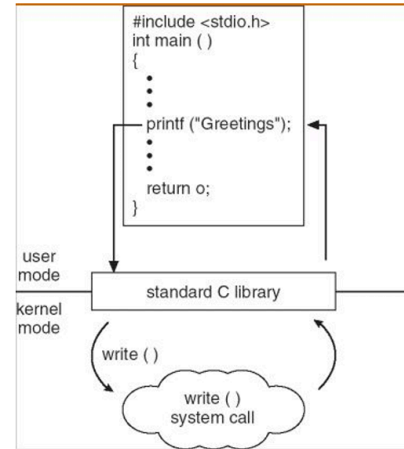
- **Mod Biti (Mode Bit):** Bu ayırım tamamen donanımın sağladığı bir "Mod Biti" ile kontrol edilir.

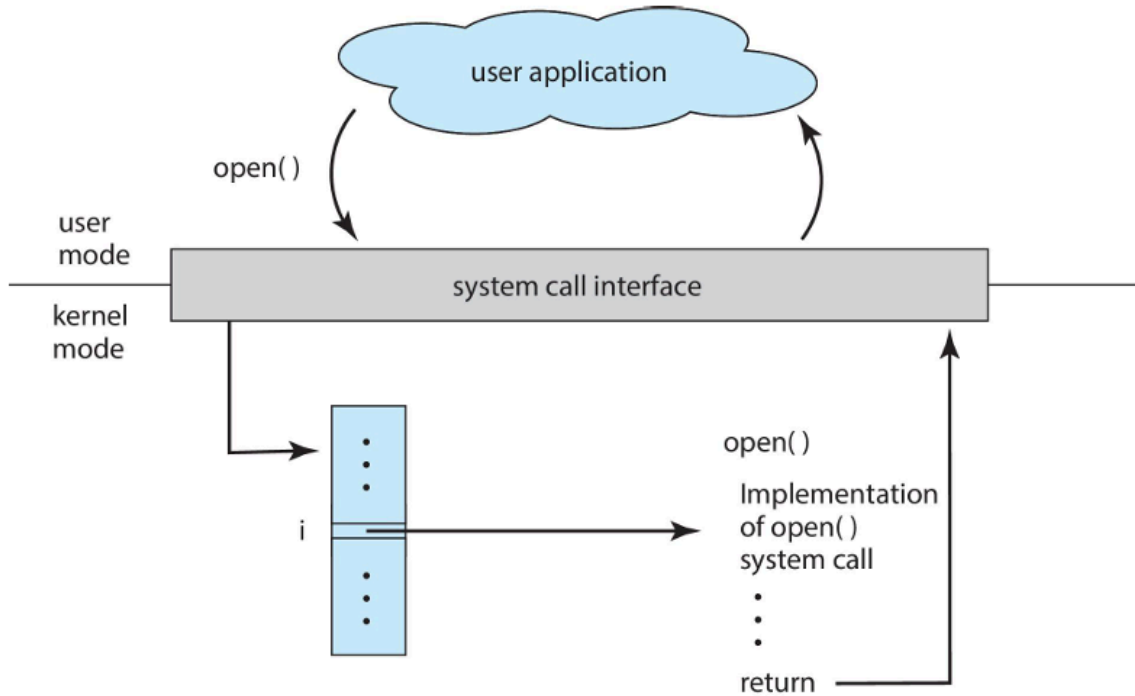
- Mod biti **1** ise sistem **kullanıcı (user)** modundadır. Mod biti **0** ise sistem tam yetkili **çekirdek (kernel)** modundadır.



Sistem Çağrılarının Yapısı ve Amacı

- Temel amaç uygulamaların ayrıcalıksız modda kendi başlarına yapamadıkları işlemleri yapabilmelerini sağlamaktır.
- C, Python veya Java'da kullandığın `printf`, `scanf`, `fgets` gibi yüksek seviyeli fonksiyonlar aslında doğrudan birer sistem çağrısı değildir; bunların hepsi kullanıcı seviyesinde (user-level) çalışan kodlardır ve asıl sistem çağrısı arayüzünün üzerine inşa edilmişlerdir.
- Sistemin gerçek çağrıları `open`, `close`, `read`, `write` gibi komutlardır. Uygulamalar, bu çağrılar için gereken argümanları (parametreleri) hazırlar ve çekirdeğe bir "trap" (tuzak/kesme) gönderir. Çekirdek bu işlemi arka planda gerçekleştirir ve sonucu uygulamaya geri döndürür.





UNIX File System Calls

- UNIX/Linux'ta her şey bir dosyadır.
- **Bağlantı Kurma (`open`)**: Bir uygulama, bir dosya veya donanımla işlem yapmak istediğinde önce onun adını kullanarak bir "açma" `open` isteği yollar. Tüm Girdi/Çıktı (I/O) işlemleri bu açık dosyalar üzerinden gerçekleşir.
- **Niyet Bildirme (Flags)**: Dosyayı açarken çekirdeğe niyetini söylemek zorundasın:
 - Sadece Okuyacağım: `O_RDONLY`
 - Sadece Yazacağım `O_WRONLY`
 - Dosya yoksa "Sıfırdan yarat": `O_CREAT`
 - İçini tamamen silip baştan yazacağım: `O_TRUNC`
 - Mevcut veriyi silmeden sonuna ekleme yapacağım: `O_APPEND`

File Descriptor

Eğer işletim sistemi bu dosyayı açmana izin verirse, sana dosyanın adını değil, **bir tam sayı (file descriptor - dosya tanımlayıcı)** geri döndürür. Artık o dosyayla ilgili okuma/yazma yapacağın zaman ismini değil, işletim sisteminin sana verdiği bu "kimlik numarasını" (FD) kullanırsın.

File Descriptor Üzerindeki Temel Yetkiler: İşletim sistemi sana o sihirli numarayı (FD) verdikten sonra, bu numara üzerinden 4 temel teorik işlem yapabilirsin

1. **Okuma (`read`):** Dosyadan veri okursun. Sistem sana kaç bayt okuyabildiğini söyler. Dosyanın sonuna gelersen 0, hata çıkarsa -1 döndürür.
2. **Yazma (`write`):** Dosyaya veri yazarsın. Yine kaç bayt yazdığını veya hata (-1) olup olmadığını döndürür.
3. **Konum Atlama (`lseek`):** Word'de yazı yazarken yanıp sönen bir imleç (cursor) vardır ya, işletim sistemi de dosyaları okurken böyle görünmez bir imleç kullanır. `lseek` komutu ile bu imleci dosyanın başına, mevcut konumuna veya en sonuna atlayabilirsin.
4. **Kapatma (`close`):** İşin bittiğinde kaynakları tüketmemek için o kimlik numarasını (FD) işletim sistemine iade edersin.

Hata Sinyali: Eğer işletim sistemi dosyayı açmana izin vermezse (örneğin yetkin yoksa), sana geçerli bir FD numarası vermek yerine `-1` döndürür. Çoğu sistem çağrısı başarısız olduğunda standart olarak -1 döndürür.

- İşletim sistemi, hatanın kodunu `errno` adında *global (genel)* bir tam sayı değişkeninin içine yazar. Örneğin, izin yoksa bu değişkene 13 ("Permission Denied") yazar. `perror` gibi fonksiyonlar da bu sayıyı alıp insanların anlayacağı metinlere çevirir.



`errno` Neden Kötü Bir Tasarımdır?

`errno`'nun global bir değişken olması, geçmişe dönük bakıldığında **kötü bir tasarım kararıdır**. Çünkü günümüzdeki modern sistemlerde aynı anda çalışan yüzlerce "iş parçacığı" (thread) vardır. İki farklı işlem aynı anda hata yaparsa, bu global değişkenin içindeki değer birbirine karışır ve modülerliği bozar.

Varsayılan File Descriptor Numaraları:

Her process için ayrı bir FD table oluşturulur ve varsayılan olarak bazı numaralarla başlar.

- **0 (Standard Input - stdin):** Varsayılan olarak klavyeye bağlıdır. Program senden bir girdi beklediğinde **0** numaralı FD'yi okur.
- **1 (Standard Output - stdout):** Varsayılan olarak ekrana (terminale) bağlıdır. `printf` gibi fonksiyonlar aslında yazdıklarını **1** numaralı FD'ye gönderir.
- **2 (Standard Error - stderr):** Hata mesajları için ayrılmış özel bir ekrandır.

Bir dosya açmak istediğinde OS sana boşta olan ilk numara olarak 3'ü verecektir.

FDs for A Single Process

OS, dosyaları yönetmek için birbiriyle bağlantılı 3 farklı dosya tutar.

1. FD Table

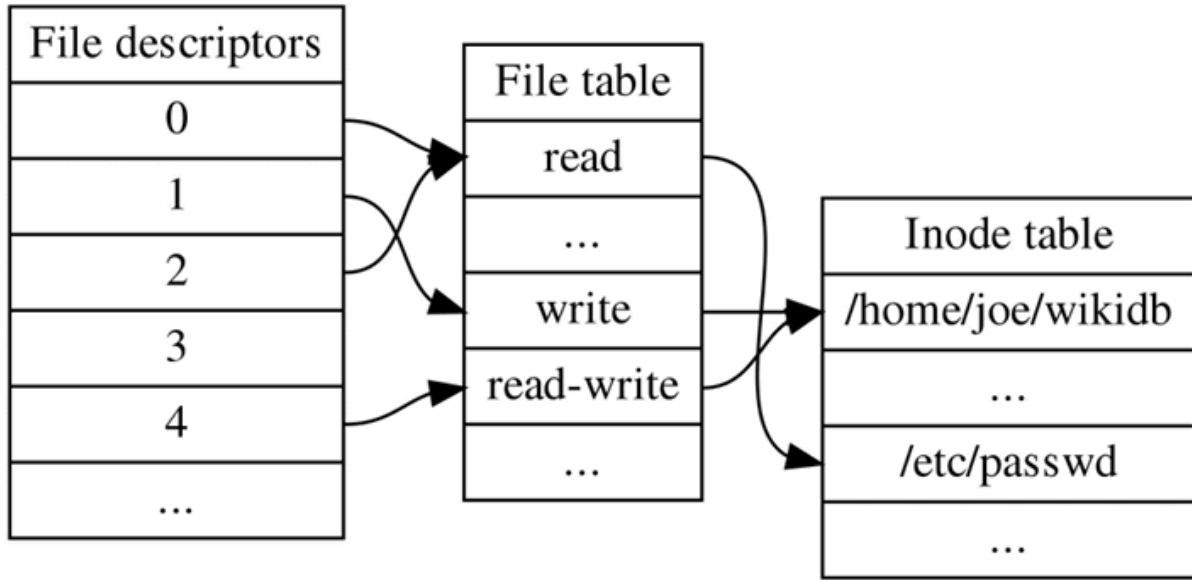
- a. Her programın kendine ait bir FD tablosu vardır.

2. File Table

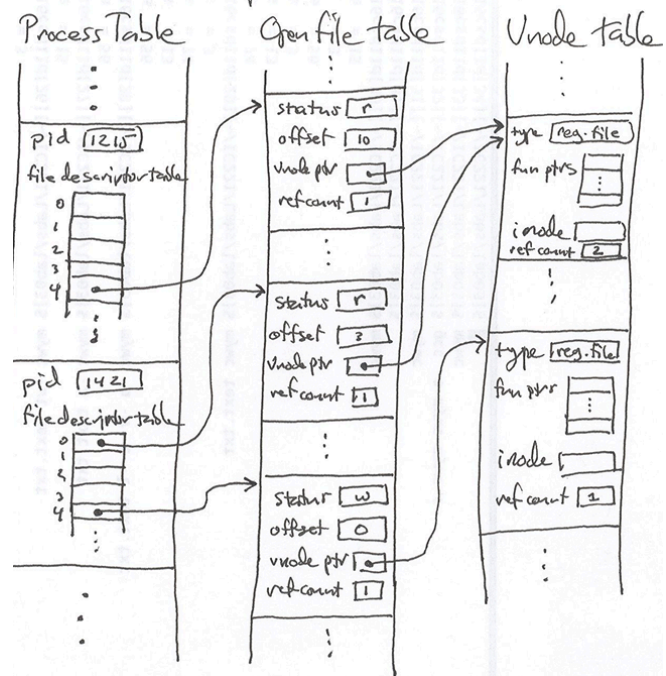
- a. Bu tablo sistem geneli için ortaktır ve OS tarafından yönetilir.
- b. Bir process bir dosyayı açtığında burada logu oluşur.
- c. Dosyanın okuma modunda mı yoksa yazma modunda mı açıldığı (access modes) ve dosyanın tam neresinde olduğumuzu gösteren "**imleç/offset**" (**file position**) tutulur.

3. Inode Table

- a. Bu da **Sistem Genelidir**. Diskteki fiziksel dosyanın ta kendisini temsil eder.
- b. Dosyanın boyutu, diskin neresinde durduğu gibi değişmez donanımsal özellikleri barındırır.



- Farklı FD'ler aynı dosya üzerinde işlem yapabilirler.
- Aynı FD farklı farklı 2 dosya üzerinde işlem yapılabilir.



İşletim Sistemi Kaynak Geri Almayı Nasıl Gerçekleştirir?

- **Timer (Timer Interrupt):** İşletim sistemi (OS), kullanıcı programına CPU'yu vermeden önce donanımsal bir zamanlayıcı kurar (örneğin "10 ms'de bir beni uyar" der).
- **Kesme (Interrupt):** Belirlenen süre dolduğunda donanım bir kesme (interrupt) üretir. Bu kesme, çalışan kullanıcı programını zorla durdurur ve CPU'nun kontrolünü anında kernel kontrolüne verir. OS bu sayede CPU'yu alıp bekleyen diğer sürece aktarabilir.
- **Güvenlik:** Sadece ayrıcalıklı modda (supervisor mode) çalışan OS bu zamanlayıcıyı ayarlayabilir. Kullanıcı programları (user code) bu sistemi hackleyemez veya kapatamaz.

Sistem Bağlamları (System Contexts)

İşlemci (CPU) çalıştığı her an belirli bir "bağlamın" (context) içindedir. O an kimin için ve hangi yetkiyle çalıştığına göre 4 temel duruma ayrılır:

1. **Kullanıcı Seviyesi (User-level):** İşlemci, ayrıcalıksız modda (user mode) sıradan bir uygulamayı (tarayıcı, oyun, yazılan bir kod vb.) çalıştırır.
2. **Çekirdek Süreç Bağlamı (Kernel Process Context):** İşlemci tam yetkili çekirdek (kernel) modundadır, ancak **belirli bir süreç (process) adına** çalışmaktadır. Uygulamanın yaptığı bir Sistem Çağrısına (syscall) veya yarattığı bir hataya yanıt veriyordur.
3. **Bağımsız Çekirdek Kodu (Kernel Code Not Associated with a Process):** İşlemci tam yetkilidir ancak belirli bir program için değil, **sistemin geneli için** çalışır. Timer interrupt (saat kesmesi) geldiğinde, donanımdan bir sinyal (device interrupt) geldiğinde veya bir süreçten diğerine geçiş (Context Switch) yaparken CPU bu bağlamdadır.
4. **Boşta (Idle):** Sistemde çalışacak hiçbir süreç kalmadığında CPU'nun güç tasarrufu için geçtiği bekleme durumudur.

Bağlam Değişiminin Maliyeti

Süreçler arasında geçiş yapmanın (context switch) getirdiği bir karmaşıklık ve performans maliyeti vardır.

Örneğin; Makinede 1 GB bellek (RAM) olduğunu ve tam 1 GB bellek isteyen 2 farklı programın aynı anda çalışmak istediğini düşün. İşletim sistemi bu iki program arasında geçiş yapabilmek için sürekli bir programı diske atıp (swap out) diğerini RAM'e çekmek zorundadır. Diskler, RAM'lere göre yaklaşık 1.000 kat daha yavaştır. Bu durumda, sürekli geçiş yapıp zaman kaybetmektense, bu iki programı aynı anda değil de sırayla (tek tek) çalıştırmak performansı çok daha fazla artırır.

Performans Arttıran Özellikler ve Önbellek

İşletim sistemi, sistemdeki yavaşlıkları maskeleyerek için donanımla birlikte bazı faydalı kuralları sömürür.

Çarpıklık (Skew):

Bir programın kodlarının %20'lik bir kısmı, toplam çalışma süresinin %80'ini oluşturur. Veya belleğin sadece %10'luk bir bölgesi, tüm bellek erişimlerinin (okuma/yazma) %90'ını karşılar.

Sonuç: İşte **Önbellek (Cache)** mantığı tamamen bu eşitsizliğin üzerine kuruludur. En çok kullanılan o %10'luk kısmı ufak ama süper hızlı bir belleğe koyarsın; geri kalan %90'lık devasa kısmı yavaş bellekte bırakırsın ve sistem dışarıdan "dev bir hızlı bellekmiş" gibi görünür.

Geçmiş, Geleceğin Aynasıdır (Temporal Locality):

Geçmişte yapılan işlemler gelecekte ne olacağını tahmin etmemizi sağlar. Eğer hızlı önbellek (cache) dolarsa ve yeni bir veriye yer açmak için eskisini silmemiz gerekirse ne yapacağız? Mantıken, geçmişte en uzun süredir kullanılmayan veriyi silmek en iyi çözümdür.